

✓ Evaluation Matrix

Install Libraries

```
%pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

```
%pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
%pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
%pip install matplotlib
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
%pip install seaborn
```

```
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.0.2)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2026.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

Library Imports

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Scikit-learn datasets & model selection
from sklearn.datasets import (make_classification, make_regression,
                              load_breast_cancer, load_diabetes)
from sklearn.model_selection import train_test_split
```

```

from sklearn.preprocessing import StandardScaler

#Classification Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

#Regression Model
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor

#Evaluation metrics - THE CORE OF THIS NOTEBOOK
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, precision_recall_curve,
    confusion_matrix, classification_report,
    mean_squared_error, mean_absolute_error, r2_score,
    ConfusionMatrixDisplay
)

```

Plotting Style

```

plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette('husl')
np.random.seed(42)

print(" All libraries loaded successfully")
print(f"Numpy: {np.__version__} | Pandas: {pd.__version__}")

```

```

All libraries loaded successfully
Numpy: 2.0.2 | Pandas: 2.2.2

```

Dataset Preparation

```

#Classification Breast Cancer Dataset
cancer = load_breast_cancer()
X_clf, y_clf = cancer.data, cancer.target #0=Malignant, 1=benign
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_clf, y_clf, test_size=0.2, random_state=42)

#Feature scaling - important for Logistic Regression and SVM
scaler_c = StandardScaler()
X_train_c = scaler_c.fit_transform(X_train_c)
X_test_c = scaler_c.fit_transform(X_test_c)

#Diabetes Dataset
diabetes = load_diabetes()
X_reg, y_reg = diabetes.data, diabetes.target
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
scaler_r = StandardScaler()
X_train_r = scaler_r.fit_transform(X_train_r)
X_test_r = scaler_r.fit_transform(X_test_r)

#Quick Data Summery
print("CLASSIFICATION DATASET (Breast Cancer)")
print(f"  Train: {X_train_c.shape} | Test: {X_test_c.shape}")
print(f"  Class balance - Malignant: {(y_clf==0).sum()} | Benign: {(y_clf==1).sum()}\n")
print("REGRESSION DATASET (Diabetes)")
print(f"  Train: {X_train_r.shape} | Test: {X_test_r.shape}")
print(f"  Target range: [{y_reg.min():.0f}, {y_reg.max():.0f}]")

```

```

CLASSIFICATION DATASET (Breast Cancer)
  Train: (455, 30) | Test: (114, 30)
  Class balance - Malignant: 212 | Benign: 357

REGRESSION DATASET (Diabetes)
  Train: (353, 10) | Test: (89, 10)
  Target range: [25, 346]

```

Student Task

```

%matplotlib inline

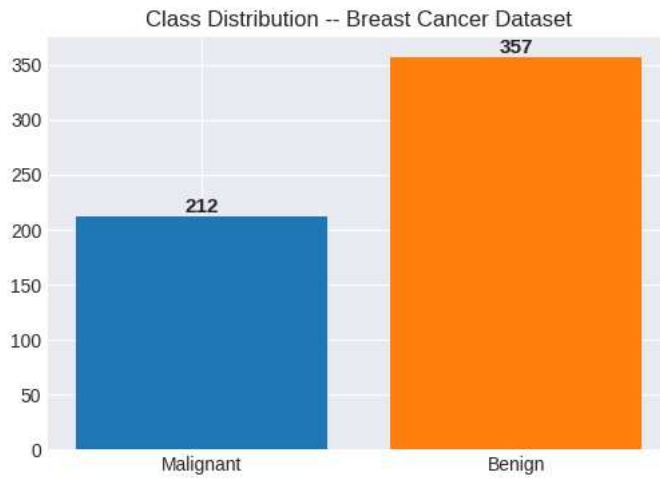
# 1. Print the first 5 feature names of breast cancer dataset
print("Feature names: ", cancer.feature_names[:5])

```

```
#2. Calculate and print Imbalance ratio
Malignant = (y_clf == 0).sum()
Benign = (y_clf == 1).sum()
Imbalance_Ratio = Malignant/Benign
print(f"Imbalance Ratio: {Imbalance_Ratio:.2f}")

#3. Plot a bar chart showing class distribution
plt.figure(figsize=(6, 4))
classes = ['Malignant', 'Benign']
count = [Malignant, Benign]
bars = plt.bar(classes, count, color = ["tab:blue", "tab:orange"])
for bar, val in zip(bars, count):
    plt.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01, f"{val}", ha='center', va='bottom', fontsize = 11, fontwe
plt.title("Class Distribution -- Breast Cancer Dataset")
plt.show()
```

Feature names: ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
'mean smoothness']
Imbalance Ratio: 0.59



Accuracy and Confusion Matrix

```
#Train three classifier for comparision

models = {
    'Logistic Regression': LogisticRegression(max_iter = 1000),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, random_state = 42)
}

results = {}

for name, model in models.items():
    model.fit(X_train_c, y_train_c)
    y_pred = model.predict(X_test_c)
    y_proba = model.predict_proba(X_test_c)[: , 1]

    results[name] = {
        'model': model,
        'y_pred': y_pred,
        'y_proba': y_proba,
        'accuracy': accuracy_score(y_test_c, y_pred),
        'cm': confusion_matrix(y_test_c, y_pred)
    }

print(f"{name}: Accuracy = {results[name]['accuracy']:.4f}")

# ----- Visualise confusion matrices side by side -----

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

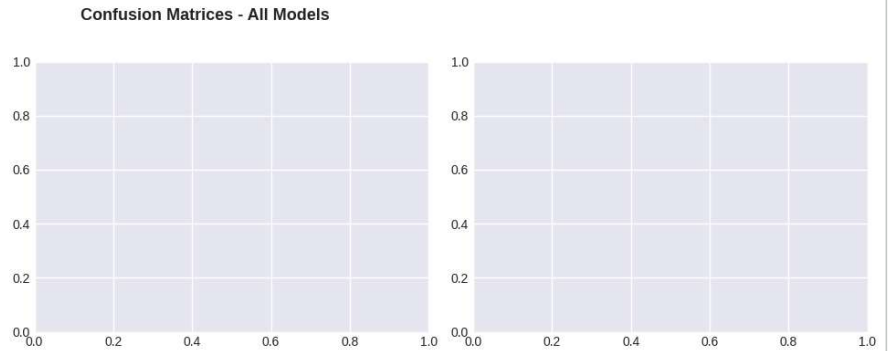
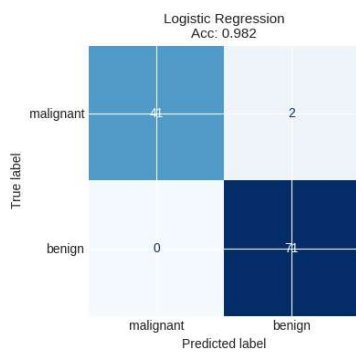
for ax, (name, res) in zip(axes, results.items()):
    disp = ConfusionMatrixDisplay(
        confusion_matrix = res['cm'],
        display_labels=cancer.target_names
    )
```

```

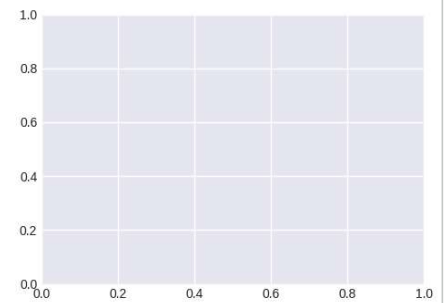
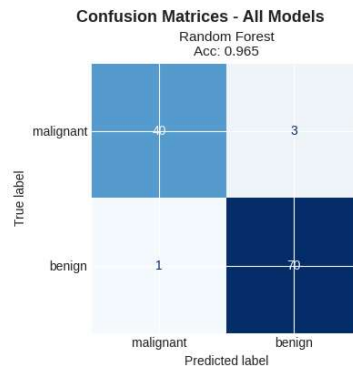
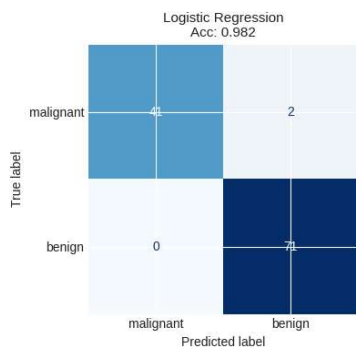
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title(f"{name}\nAcc: {res['accuracy']:.3f}", fontsize=11)
plt.tight_layout()
plt.suptitle("Confusion Matrices - All Models", y=1.02, fontsize=13, fontweight='bold')
plt.show()

```

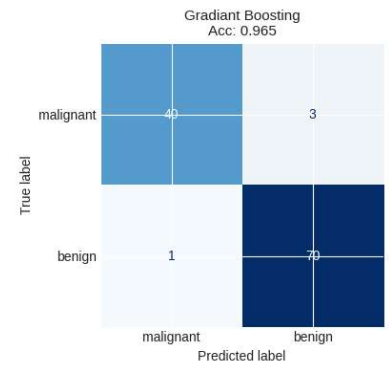
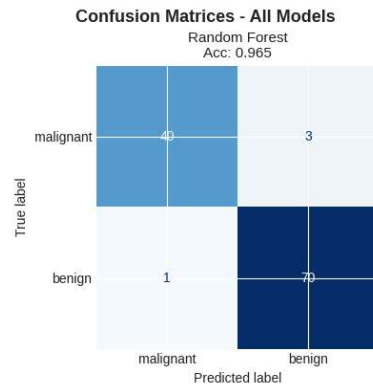
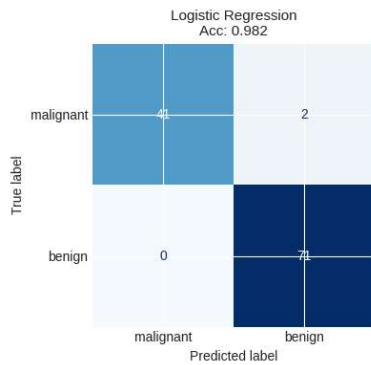
Logistic Regression: Accuracy = 0.9825



Random Forest: Accuracy = 0.9649



Gradient Boosting: Accuracy = 0.9649



Student Work

#1. Pick the Logistic Regression model's confusion matrix

```
Log_Reg_CN = results['Logistic Regression']['cm']
print(Log_Reg_CN)

#2. Extract TP, FP, TN, FN
TP = Log_Reg_CN[1][1]
TN = Log_Reg_CN[0][0]
FP = Log_Reg_CN[0][1]
FN = Log_Reg_CN[1][0]
print(f"TP: {TP}, TN: {TN}, FP: {FP}, FN: {FN}")

#3. Calculate accuracy from scratch (without sklearn) and verify it matches
manual_acc = (TP+TN)/(TP+TN+FP+FN)
print(f"Manual Accuracy: {manual_acc:.4f}")
sklearn_acc = results['Logistic Regression']['accuracy']
print(f"Sklearn Accuracy: {sklearn_acc:.4f}")
```



```
[[41  2]
 [ 0 71]]
TP: 71, TN: 41, FP: 2, FN: 0
Manual Accuracy: 0.9825
Sklearn Accuracy: 0.9825
```